

GPI-8510 신규 PCB 펌웨어 개발 7일 절차서

BSP 추상화 기반 재사용 개발 프로세스 — "3개월 → 7일" 단축 로드맵

대상 프로젝트 : GPI-8510 (축산 사일로 사료 계량기) **MCU** : STM32F103ZET6 (Cortex-M3)

개발환경 : Keil MDK uVision + StdPeriph **보조도구** : VS Code + Claude Code

인코딩 : CP949 / CRLF **작성일** : 2026-07-15

이 문서의 목적 — 기존 GPI-8510 펌웨어를 BSP·드라이버·서비스·앱 4계층으로 정리해 두면, 새 PCB가 나올 때마다 **핀맵 헤더 1개와 앱 로직만** 새로 작성하면 됩니다. 본 절차서는 그 준비(리팩토링)와 실제 신규 PCB 대응을 **7일 일정**으로 나눈 실행 계획입니다.

1. 전체 개요 — 왜 7일이 가능한가

기존에 신규 PCB 펌웨어가 3개월 걸린 이유는 **핀 번호와 하드웨어 제어 코드가 8,000줄 규모의 소스 전체에 흩어져 있어**, 새 보드마다 전 소스를 수색·수정하고 회로도 와 대조하는 작업을 반복했기 때문입니다. 이를 계층 구조로 한 번 정리해 두면, 변하는 부분(핀맵·앱 로직)과 변하지 않는 부분(드라이버·서비스)이 분리되어 재사용이 가능해집니다.

구분	기존 방식 (3개월)	BSP 도입 후 (7일)
핀 변경 대응	전체 소스 수색·수정	헤더 1개(<code>board_xxxx.h</code>) 수정
드라이버 코드	매번 새로 작성/복붙	driver 계층 그대로 재사용
통신·큐·파싱	매번 재검증	service 계층 그대로 재사용
신규 작성 범위	거의 전부	핀맵 + 앱 로직만
회로도 ↔ 코드 대조	수작업, 누락 잦음	헤더가 곧 회로도 문서

전제 조건 — 본 7일 일정은 **Phase 1 (BSP 헤더 `board_gpi8510.h` 적용)**이 **완료된 상태**를 출발점으로 합니다. 현재 모든 파일에 매크로 적용 후 컴파일 테스트 단계에 계셨으므로, 이 절차서는 **그 검증(Day 1)부터** 시작합니다.

2. 리팩토링 계층 구조 (목표 형태)

계층	폴더	담당	신규 PCB 시
app	app/	배출량 계산, 전송 정책, 화면 구성 (기존 <code>Agent_*</code> 태스크)	일부 신규 작성
service	service/	통신 프로토콜, 큐, 파싱, 무게 필터, CDMA 상태머신	재사용
driver	driver/	ADC, LCD, UART, EEPROM, Flash, RTC 드라이버	재사용
bsp	bsp/	핀맵, 클럭 정의 (<code>board_xxxx.h</code>)	헤더만 교체

3. 7일 상세 일정

Day 1 — BSP 적용 검증 및 기준선 확보

검증

목표: 모든 파일에 적용한 `board_gpi8510.h` 매크로가 정상 동작함을 하드웨어로 최종 확인하고, 이후 작업의 "기준점(baseline)"을 확정한다.

업무 내용

- Keil에서 전체 빌드 → **Program Size(Code/RO/RW/ZI)를 BSP 적용 이전 값과 비교** (Code≈46KB, RO≈48KB, RW≈1.3KB, ZI≈23KB 기준)
- 실제 장비에 다운로드 후 핵심 기능 순차 점검: 부저 → LED → 키 입력 → LCD 표시 → 무게 측정(ADC) → RS-485 → 모뎀 통신
- 이전 대비 **바이너리 크기·동작이 동일한지** 확인 (BSP는 이름만 바꾼 것이므로 동작은 100% 같아야 정상)
- 이상 없으면 이 커밋을 `baseline-bsp-ok` 태그로 저장 (되돌릴 기준점)

주의 — 만약 크기나 동작이 달라졌다면, 매크로 치환 중 핀 번호가 틀린 곳이 있다는 신호입니다. 파일 단위로 되돌려가며 원인을 찾으세요. 여기서 완벽히 잡고 넘어가야 이후가 안전합니다.

Day 2 — driver 계층 추출 (1): 안전 모듈

Phase 2

목표: 하드웨어 드라이버를 `driver/` 로 분리하기 시작한다. 망가져도 위험이 적은 모듈부터.

업무 내용

- 대상(안전 순): `buzzer` → `led` → `key` → `rtc` 순으로 각 드라이버 파일을 `driver/` 폴더로 이동
- 순수 이동만** — 함수 내용은 바꾸지 않고 위치만 옮기고 `extern` 선언 정리
- 한 모듈 이동 → Keil 빌드 → 하드웨어 확인 → 다음 모듈 (한 번에 하나씩)

Claude Code 프롬프트 예시:

```
buzzer 관련 함수를 driver/buzzer.c 로 순수 이동해줘.
동작은 바꾸지 말고 위치만 옮기고, 필요한 extern 선언을 정리해줘.
CP949 인코딩 유지, StdPeriph API만 사용, 워치독 리로드 경로 유지.
```

Day 3 — driver 계층 추출 (2): 민감 모듈

Phase 2

목표: 캘리브레이션·화면·통신에 직결된 민감 드라이버를 분리한다. 검증 강도를 높인다.

업무 내용

- 대상: `uart` (USART1~5) → `eeeprom` → `flash` (W25Q128) → `lcd` → `adc` (CS5555/ADS1251) 순
- 각 모듈 이동 후 반드시 실측 검증: UART는 통신 로그, EEPROM은 factor 읽기/쓰기 값 대조, ADC는 무게 표시값 확인

절대 금지 — EEPROM 캘리브레이션 변수(`gnfFactor` , `v_zero` , `v_span`)는 **값을 건드리지 말 것**. 코드 위치만 옮기고 저장 데이터는 그대로 두세요. 현장 장비 정밀도가 걸린 부분입니다. FSMC 활성 상태에서 EEPROM 접근 코드 작성 금지.

Day 4 — service 계층 추출

Phase 2

목표: 통신·큐·파싱 등 미들웨어를 `service/` 로 분리한다. 이 계층이 신규 PCB에서 그대로 재사용되는 핵심 자산이 된다.

업무 내용

- 대상: 프로토콜 파싱(`#...@` , `$....@`), CDMA 상태머신(`iSendStage` , `CDMA_STAGE_*`), 송신 큐(`CdmaSendQueue`), 무게 필터
- 인터럽트 핸들러(`stm32f10x_it.c`)와 공유되는 변수는 **volatile 유지 확인**
- 시간 관련 로직은 `DEF_1_SEC` / `DEF_1_MIN` 기반 유지 (ms 직접 계산 금지, 10ms 틱 체계)

Day 5 — app 계층 정리 및 표준 프롬프트 세트 구축

Phase 3

목표: 비즈니스 로직을 `app/` 으로 정리하고, 신규 PCB 개발용 "표준 작업 프롬프트 세트"를 완성한다.

업무 내용

- 기존 `Agent_WeightDisplay_Task()` , `Agent_DischargeCalc_Task()` 등을 `app/` 계층 파일로 정리
- `main.c` 는 초기화 + 태스크 호출 골격만 남기는 방향으로 슬림화 (한 번에 하나씩, 동작 변경 금지)
- CLAUDE.md에 "신규 보드 개발 표준 절차" 섹션 추가** — 아래 5장의 프롬프트 세트를 문서화

Day 6 — 신규 PCB 대응 실전 (핀맵 + 앱)

신규

목표: 정리된 구조 위에서 실제 신규 PCB 펌웨어를 만든다. 이 날이 "3개월 → 7일"의 실증 단계.

업무 내용

- 신규 회로도 확보 → `board_gpi8510.h` 를 복사해 `board_신규.h` 생성 → **바뀐 핀만 수정**

- driver / service 계층은 **손대지 않고 그대로 링크**
- 신규 보드에서 달라진 요구사항(추가 센서, 다른 화면 구성 등)만 **app/** 에 신규 작성
- 빌드 → 신규 장비에 다운로드 → 기본 기능(전원/부저/LED/키/화면) 우선 점검

Claude Code 프롬프트 예시:

```
새 회로도들 기준으로 board_신규.h의 핀 정의를 수정하려고 해.  
기존 board_gpi8510.h를 복사한 상태야. 아래 회로도 신호명과 핀을  
대조해서 바뀐 것만 알려주고 수정해줘. driver/service는 건드리지 마.
```

Day 7 — 통합 검증 및 현장 준비

검증

목표 : 신규 PCB 펌웨어를 실사용 조건에서 종합 검증하고 출하 가능 상태로 만든다.

업무 내용

- 전 기능 통합 테스트: 무게 측정 정확도, 캘리브레이션(영점/스팬), 모뎀 전송 주기(1시간/10분), RS-485, RTC 시각
- 워치독 리셋 무발생 장시간 구동 테스트 (긴 블로킹 없음 확인)
- IAP 원격 펌웨어 업데이트 동작 확인 (백터 오프셋 **0x3000** 유지 여부)
- 버전 이력 갱신(main.c 상단, def.h) → 최종 빌드 → 릴리스 태그 저장

4. 일정 요약표

Day	단계	핵심 업무	완료 판정 기준
1	검증	BSP 적용 최종 검증, 기준선 확보	바이너리 크기·동작 이전과 동일
2	Phase 2	driver 추출 — 안전 모듈(부저/LED/키/RTC)	각 모듈 이동 후 하드웨어 정상
3	Phase 2	driver 추출 — 민감 모듈(UART/EEPROM/LCD/ADC)	무게·factor 값 이전과 일치
4	Phase 2	service 추출 — 프로토콜/큐/상태머신	통신 송수신 정상, volatile 유지
5	Phase 3	app 정리 + 표준 프롬프트 세트 문서화	main.c 슬림화, CLAUDE.md 갱신
6	신규	신규 PCB 핀맵 헤더 + 앱 로직 작성	신규 장비 기본 기능 동작
7	검증	통합 검증 + IAP + 현장 준비	전 기능 통과, 릴리스 태그

5. 신규 PCB 개발 표준 프롬프트 세트

Day 5에서 CLAUDE.md에 문서화할 재사용 프롬프트입니다. 신규 보드가 나올 때마다 이 순서로 사용하면 됩니다.

단계	프롬프트
① 핀맵	회로도 신호명 기준으로 board_신규.h의 #define을 새 보드에 맞게 수정해줘. 기존 대비 바뀐 핀만 표로 보여줘.
② 분석	main.c의 XXXX-YYYY 라인 함수를 분석하고 어떤 하드웨어를 건드리는지 설명해줘.
③ 드라이버	CS5555 비트뱅잉 읽기 코드를 driver/adcs5555.c로 추출해줘. 채널을 파라미터로 받는 구조로.
④ 앱	신규 보드의 추가 요구사항 (예: 센서 1개 추가)만 app/계층에 신규 작성해줘. 기존 태스크 패턴을 따라줘.
⑤ 검증	이 수정이 StdPeriph API와 C89 문법을 준수하고 워치독 경로를 유지하는지 검토해줘.

6. 매일 지켜야 할 안전 규칙 (체크리스트)

✓	규칙
☐	한 번에 한 모듈씩만 변경하고, 매번 Keil 빌드로 확인한다
☐	함수 이동 시 동작 변경 금지 (순수 이동 → 빌드 → 정리 순서)
☐	CP949 인코딩 유지 (UTF-8 변환 금지 — 한글 주석 깨짐)
☐	StdPeriph API만 사용 (HAL/LL/CubeMX 금지)
☐	C89 문법 유지, 메인루프에 긴 블로킹 삽입 금지 (워치독 26초 리셋)
☐	<code>IWDG_ReloadCounter()</code> 호출 경로 제거 금지
☐	NVIC 벡터 오프셋 <code>0x3000</code> 수정 금지 (IAP 부트로더 영역)
☐	캘리브레이션 변수(<code>gnfFactor</code> / <code>v_zero</code> / <code>v_span</code>) 사용자 확인 없이 변경 금지
☐	전역변수·함수 이름 일괄 변경은 반드시 사용자 승인 후
☐	작업 후 Program Size(Code/RO/RW/ZI)를 이전과 비교한다

핵심 원칙 한 줄 요약 — "변하지 않는 것(driver·service)은 재사용하고, 변하는 것(핀맵·앱)만 새로 만든다. 그리고 무엇을 바꾸든 한 번에 하나씩, 빌드로 확인하며 나아간다."